

2 Graph Theory

Graph Theory was studied by mathematicians long before Network Science took off in the late 90s. Mathematicians were interested in studying particular families of graphs with certain restrictions or symmetries, like regular graphs (all nodes with the same degree) or planar ones (can be drawn without edge crossings). A famous problem in graph theory is the four-colour theorem - which says the fewest possible colours required to colour a map such that no two neighbouring countries are coloured with the same colour is 4.

Network Science on the other hand is about understanding the real world through network data. For example, how a disease spreads on a contact network and the most effective immunization strategies to stop it. Real networks generally don't have nice symmetries and often have properties very different than the types of graphs that traditionally interest mathematicians.

However, graphs and networks are the same mathematical objects and there is naturally a lot of overlap between Graph Theory and Network Science. This connection to pure maths also brings a lot of precision to the basic definitions. This section will terminology we will use for the rest of the module.

Reading

[1] **Chapter 2**, [2] **Chapter 6**.

2.1 Basic notation

A network/graph $G = [V, E]$ is a mathematical object consisting of a set of nodes/vertices $V = \{v_1, v_2, \dots, v_N\}$ and a collection of links/edges $E = [e_1, e_2, \dots, e_L]$ which are pairs of nodes/vertices. The different terms separated by a slash: network/graph, link/edge, node/vertex, are synonyms. The ones on the right (graph, edge, vertex) are more typical in mathematical graph theory and the ones on the left (network, link, node) are common in network science. I will use them interchangeably but generally prefer the network, link, node description.



Figure 1: Left: undirected network. Right: directed network.

If we define a link as an unordered pair of nodes, $e = \{v_a, v_b\}$, then we have an **undirected network**. If we define them as ordered pairs, $e = (v_a, v_b)$ we get a **directed network**. In either case we say a is connected/linked to b via the edge e . For convenience we will mostly use the node labels to specify a node or edge e.g. $e = \{a, b\}$ links a and b . Figure 1 shows a **graph drawing** of the undirected network with vertex set $V = \{a, b, c, d, e, f\}$ and edge set

$$E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{c, d\}, \{d, e\}, \{d, f\}\}$$

and the directed graph with the same vertex set but with edge set

$$E = \{(a, c), (b, c), (c, a), (d, b), (e, d), (f, d)\}$$

The number of nodes will be denoted $N = |V|$. The number of links will be denoted $L = |E|$.



Figure 2: Left: simple network. Right: non-simple network.

In these lectures we will mostly consider undirected networks, which are easier to analyse. The definition above still allows us to have edges and nodes which we may or may not want to consider.

- An **orphan** or **isolated** node is one that has no edges connected to it.

- A **multi-edge** arises when we have more than one edge connecting the same nodes.
- A **self-edge** or **self-loop** is an edge that connects a node to itself $e = \{a, a\}$

In Figure 2, g is an orphan node, there is a 3-edge between a and b , a 2-edge between d and f and a self-loop on node e . Networks without multi-edges or self-loops are called **simple** networks. Orphaned nodes usually don't cause problems, so we generally allow them. Depending on the application we may or may not want to allow multi-edges and self-loops. Self-loops generally cause the most problems so it can be convenient to delete or ignore them.

2.2 Adjacency matrix

In graph theory one often works with this abstract definition of a network, however in network science it is common to rely on the **adjacency matrix**, which is an alternative way to define a network. For simple graphs the elements of the adjacency matrix, A are

$$A_{ij} = \begin{cases} 1 & \text{if } i \text{ is connected to } j \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

For the left network in Figure 2 (where the nodes are listed in alphabetical order)

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Note, for any undirected network this matrix is symmetric, a connection between i and j implies a connection between j and i . For non-simple networks we can represent multi-edges by putting the multiplicity of the edge in the adjacency matrix instead of 1. For self-loops we put a 2 on the corresponding diagonal. Why 2 and not 1? The maths works out much nicer this way. You can think of the element in the adjacency matrix as counting the number of **stubs**, coming out of the node, where a stub is what we would get from cutting the links and leaving the ends dangling from the nodes. Cutting a self loop would leave two stubs, hence we count self-loops as 2 in the adjacency matrix.

2.3 Degree

Going back to simple networks, if we count up all the non-zero terms in the adjacency matrix we get twice the number of edges

$$2L = \sum_{ij}^N A_{ij} \quad (2)$$

where the factor 2 comes from counting every edge twice, once from i to j and once from j to i . A very important property of a node is its **degree**, which is the number of edges connected to the node. We can obtain the degree of a node by summing up the row or column in the adjacency matrix corresponding to that node

$$k_i = \sum_j^N A_{ij} \quad (3)$$

Combining these two equations gives

$$2L = \sum_i k_i \quad (4)$$

The average or **mean degree** is

$$\langle k \rangle = \frac{1}{N} \sum_i k_i = \frac{2L}{N} \quad (5)$$

This turns out not to be very useful as a summary of real world networks, which tend to have quite a wide range of degrees.

2.4 Density

The maximum possible number of edges in a simple network with N nodes would have each of the N nodes connected to all of the other $N - 1$ nodes or

$$L_{max} = \binom{N}{2} = \frac{N(N-1)}{2}$$

Such a network is called a **complete graph** on N nodes, often denoted K_N . When a complete graph is part of a larger network it is called a **clique**. The nodes $\{a, b, c\}$ form a clique in the simple graph in Figure 2. In general, any $U \subset V$ together with edges from E which only contain elements from

U forms a **subgraph** of G . So a clique is a subgraph with N vertices is identical or **isomorphic** to the complete graph K_N .

It can be useful to compare the maximum possible number of links between N nodes with the number of observed links

$$\rho = \frac{2L}{2\binom{N}{2}} = \frac{L}{N(N-1)} \quad (6)$$

which is called the **connectance** or **density**. For most real world networks the density is quite low i.e. most of the links which could be there are not.

2.5 Degree distribution

The **degree distribution** is the probability that a randomly selected node has degree k . We compute it as

$$p_k = \frac{N_k}{N} \quad (7)$$

Where N_k is the number of nodes with degree k .

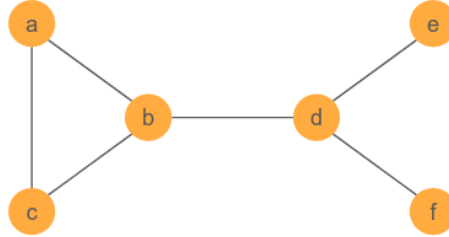


Figure 3: A network.

The degrees in the network in Figure 3 are

$$k_a, k_b, k_c, k_d, k_e, k_f = 2, 3, 2, 3, 1, 1$$

The degree distribution is

$$p_1 = 2$$

$$p_2 = 2$$

$$p_3 = 2$$

and $p_k = 0$ for any other value of k . Knowing the degree distribution lets us calculate different degree statistics such as the moments

$$\langle k^n \rangle = \frac{1}{N} \sum_i^N k^n p_k$$

or the variance

$$\sigma^2 = \langle k^2 \rangle - \langle k \rangle^2$$

Real networks often have **fat tailed** or **long tailed** degree distributions. These are distributions where the probability of having a really high degree node decays slower than exponentially, unlike, say, a Gaussian. This can lead to very large terms k^n in the sums above, such that, in the limit of large N higher moments can even diverge to infinity.

2.6 Weighted networks

Edges in real networks often have some cost or value associated with them. We represent these as **weighted networks** where the **weight matrix** W with elements W_{ij} carries the associated cost of a link and we put a 0 when there is no connection.

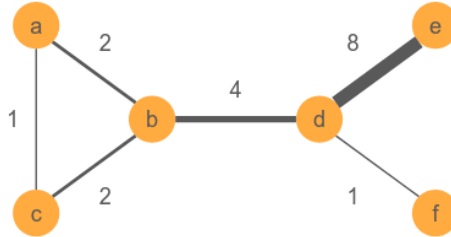


Figure 4: Weighted network.

The weight matrix of the network in Figure 4 is

$$W = \begin{pmatrix} 0 & 2 & 1 & 0 & 0 & 0 \\ 2 & 0 & 2 & 4 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 8 & 1 \\ 0 & 0 & 0 & 8 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

It is sometimes useful to distinguish between the adjacency matrix, which records if two nodes are connected and the weight matrix, which records the value of the connections, but usually the weight matrix is the important thing to consider in the study of weighted networks. In this context, it can sometimes be useful to define a weighted degree by

$$k_i = \sum_j W_{ij} \quad (8)$$

We in the next sections we will see many quantities defined in terms of the adjacency matrix. In general the same approach, replacing A_{ij} with W_{ij} , will give the corresponding metric for a weighted network.

2.7 Directed networks

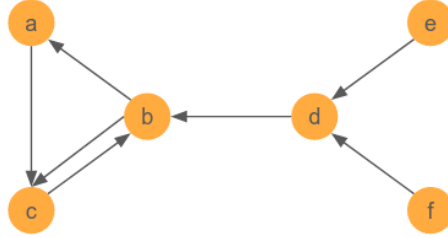


Figure 5: Directed network.

As mentioned, a directed network is one where the order of the pair matters, so a link from a to b is different from and does not imply a link from b to a . We usually denote this in the graph drawing by putting an arrow on the corresponding link. A directed network's adjacency matrix is not necessarily symmetric, $A_{ij} \neq A_{ji}$,

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

is the adjacency matrix of the network in Figure 5.

For directed networks we have an **in-degree** and an **out-degree**, counting the number of incoming and outgoing links

$$k_i^{in} = \sum_j A_{ji} \quad (9)$$

$$k_j^{out} = \sum_i A_{ij} \quad (10)$$

and corresponding in and out degree statistics. Sometimes the direction of the link is not relevant for the questions we are asking about a network, so we may use metrics and algorithms developed for undirected networks on directed ones by simply forgetting about the arrows, or adding all the reverse arrows (as in the connection between b and c in Figure 5), which is equivalent.

2.8 Bipartite networks

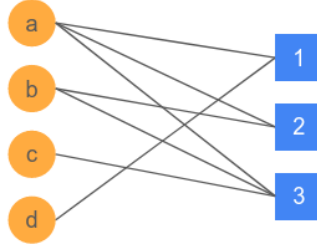


Figure 6: Bipartite network.

Another special type of network which frequently comes up in applications is a **bipartite network**. In a bipartite network the vertex set V can be split into two disjoint subsets V_1, V_2 where $V = V_1 \cup V_2$ and $V_1 \cap V_2 = \emptyset$. Figure 6 shows an example. V_1 and V_2 are called left and right or top and bottom parts and often represent different types of entity for example authors and papers or ingredients and recipes. In Figure 6 we use letters for the left part and numbers for the right as labels to emphasise this difference.

For a bipartite network the adjacency matrix has a special form. If there are N_1 nodes in V_1 and N_2 in V_2 (where $N = N_1 + N_2$ of course) then if the nodes are labelled such that the first N_1 labels are in V_1 and the rest in V_2

$$A = \begin{pmatrix} 0_{N_1 \times N_1} & B \\ B^T & 0_{N_2 \times N_2} \end{pmatrix} \quad (11)$$

Where $0_{N \times M}$ is a $N \times M$ matrix of all zeros and B is a $N_1 \times N_2$ matrix called the **biadjacency matrix**. This is often more useful than the adjacency matrix for bipartite networks, since it contains all the information about the network with less redundancy than the adjacency matrix. We have

$$\sum_{ij}^{N_1, N_2} B_{ij} = L$$

and it can be useful to define left and right degrees using the biadjacency matrix

$$l_i = \sum_j B_{ij}$$

$$r_j = \sum_i B_{ij}$$

A projection is a way of transforming a bipartite network that turns it into a (weighted) network on just one type of node (which we might call **unipartite** to distinguish from bipartite). There are several ways to do this, one common one is

$$A_{ij} = (1 - \delta_{ij}) \sum_k B_{ik} B_{jk} \quad (12)$$

The symbol δ_{ij} is called the **Kronecker delta**,

$$\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \quad (13)$$

so $(1 - \delta_{ij})$ is zero when $i = j$ and has the effect of zeroing all the self edges. The sum just counts up all the nodes that i and j both connect to,

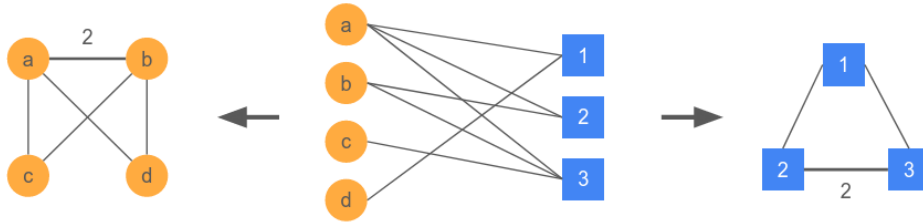


Figure 7: Bipartite network projections.

Figure 7 shows the left and right projections of a small bipartite network. Note the projections are weighted. There are different versions which keep self-loops or throw away the weights. The main thing to remember is that projections of bipartite graphs often produce very dense networks.

2.9 Paths

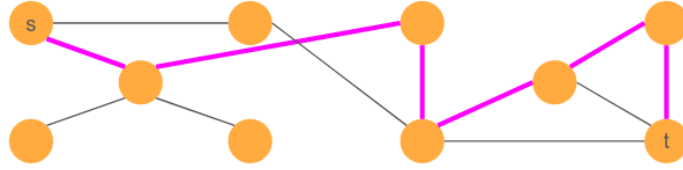


Figure 8: A path from s to t .

A path is a sequence of vertices where each pair of adjacent vertices in the path is connected by an edge i.e. we go from s to t following the links in the network, see Figure 8. The length of the path is the number of links it traverses. For a weighted network, we can add the weight of each link to compute the total cost of the path. For a directed network we have to follow the direction of the links. If the length (or cost) of a path between i and j is d_{ij} then in an undirected network $d_{ij} = d_{ji}$ but in a directed network $d_{ij} \neq d_{ji}$. A **shortest path** between nodes is just as it sounds, a path with minimal distance among all paths connecting the two nodes.

We can use the adjacency matrix to count paths of each length.

1. The length 1 paths are given by the non-zero entries of the adjacency matrix.
2. If there is a path of length 2 between vertices i and j then it must go via some intermediate vertex k such that $A_{ik}A_{kj} = 1$. We can sum over all intermediate nodes

$$\sum_k A_{ik}A_{kj}$$

Terms in the sum will only be non-zero when there is a link from i to some node k and then to j . This formula is identical to the formula for matrix multiplication

$$A_{ij}^2 = \sum_k A_{ik}A_{kj}$$

That is, elements of the square of the adjacency matrix count the number of paths of length 2 between i and j .

3. Generalising, A_{ij}^d counts the paths of length d between i and j .

The **network diameter** D is the longest shortest path. The **average path length** is

$$\langle d \rangle = \frac{1}{N(N-1)} \sum_{ij, i \neq j}^N d_{ij} \quad (14)$$

Name	Description
Path	A sequence of nodes connected by edges
Path length	The number of edges on a path
Shortest Path	A path between two nodes with minimal length.
Cycle	A path that begins and ends at the same node.
Eulerian Path	A path visiting every edge in the network.
Hamiltonian Path	A path visiting every vertex in the network.

The table above collects some path terminology. It is a somewhat surprising fact that Eulerian paths are quite easy to characterise and Hamiltonian paths are quite difficult. These kinds of things are usually studied in classical graph theory and we won't worry about them much in this course.

2.10 Breadth first search

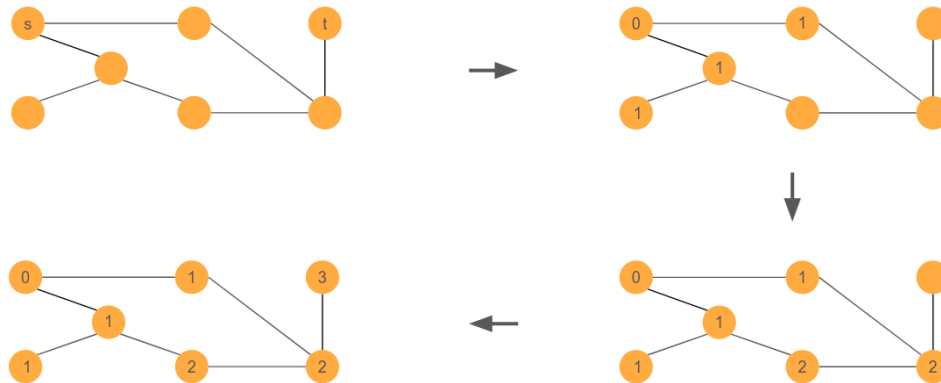


Figure 9: BFS starting from s .

There is a simple and ubiquitous algorithm to find shortest paths called **breadth first search** (BFS).

1. Label the start node, s , with a 0, label the nodes directly connected to it with a 1 and put them in a queue.
2. At each step pop the node from the front of the queue, labelled n , label its unlabelled neighbours $n + 1$ and put them in the queue.
3. Repeat step 2 until the target node is reached or the queue is empty.
4. The distance between s and the target is given by the label of the target. If the target has no label the distance is infinite, meaning there is no path between s and the target.

Figure 9 shows the BFS algorithm at a few points along the process.

2.11 Connected components

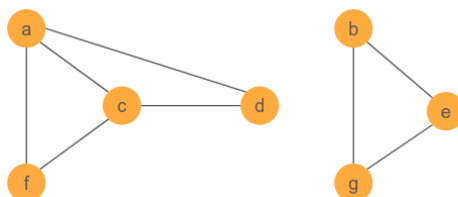


Figure 10: A disconnected network.

There are many networks like the one in Figure 10 which are split into two parts or **components**. Formally, we say a network is **disconnected** into components G_1 and G_2 if there is no path from nodes in G_1 to nodes in G_2 . If there is a path from every node to every other one then G is **connected**.

For disconnected networks it is often necessary to consider the components separately. BFS gives a simple way to find them. Starting from any node, perform the BFS algorithm labelling every visited node with a 1 in this case. If the total number of labelled nodes is N then the network is connected. If not, then pick an unlabelled node and repeat, this time using the label 2. When all the nodes have been labelled, the last label gives the number of components and nodes with the same label are in the same component.

When a network is disconnected its adjacency matrix can be brought into block-diagonal form by relabelling the nodes so nodes in the same component have sequential indices. Using alphabetical order for the rows and columns of the adjacency matrix in Figure 10 gives

$$A = \begin{pmatrix} 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

We can relabel the nodes as below

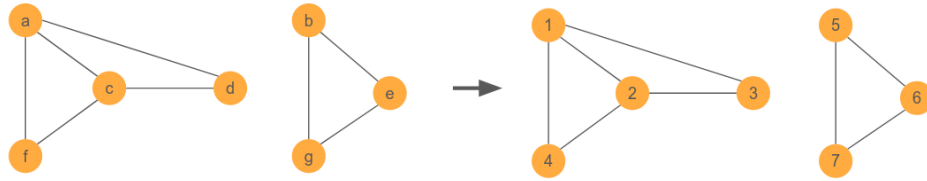


Figure 11: Disconnected network relabelled.

With these labels the adjacency matrix is

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} A_1 & 0_{4 \times 4} \\ 0_{3 \times 3} & A_2 \end{pmatrix}$$

A matrix like this is said to be **block diagonal**. Finding labellings of networks that make the adjacency matrix block diagonal, or almost block diagonal, is one of the major problems in network science.

2.12 Directed paths

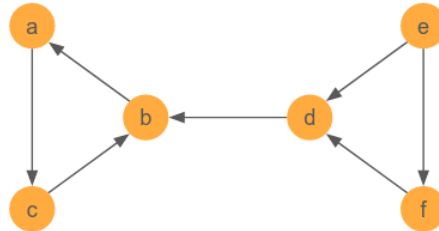


Figure 12: Directed network.

For a directed network we have different notions of connectivity. If a directed network is connected when we ignore the direction of the edges then we say the network is **weakly connected**. However, in a directed network we can have a path from i to j but no path from j to i . In the network of Figure 12 there is a path from e to a but no path from a to e . If there is a *directed* path between every pair of vertices then the network is said to be **strongly connected**.

We might only be concerned about paths going in one direction, for example in a network of infections where having the disease confers immunity (or death), links back to the original carrier aren't important. If there is a path starting from a then a network is **out-connected** from a if there is a path from a to any other vertex. In Figure 12 the network is out-connected from e (and only e). We can similarly define an **in-connected** component, where there is a path *from* any node to a . Figure 12 is in-connected to a , b and c .

References

- [1] Márton Pósfai and Albert-László Barabási. *Network science*. Cambridge University Press, 2016.
- [2] Mark Newman. *Networks*. Oxford university press, 2018.